

NavDR: Navigation Dead Reckoning System

Comprehensive Project Documentation & Progress Report

1. The Problem Statement

Modern smartphone navigation systems rely entirely on Global Navigation Satellite Systems (GNSS/GPS) to determine user position and velocity. However, GNSS signals are inherently fragile. They suffer from severe degradation in challenging environments such as:

- **Urban Canyons:** High-rise buildings block line-of-sight to satellites, and signal reflections off skyscrapers cause "multipath" errors, resulting in the user's location jumping wildly across the map.
- **Tunnels & Underpasses:** Complete loss of signal causes navigation apps to freeze or blindly interpolate straight lines, leaving drivers without critical exit instructions.
- **Heavy Foliage & Bad Weather:** Attenuates signals, reducing accuracy.

When GPS fails, conventional navigation apps are left completely blind. Classical Dead Reckoning (integrating accelerometer data twice to find position) mathematically explodes due to the inherent noise and bias in cheap smartphone sensors.

2. The Proposed Solution (NavDR)

NavDR is a pure software solution designed to bridge GNSS outages using the smartphone's built-in Inertial Measurement Unit (IMU). Instead of relying on flawed classical physics integration, NavDR leverages Machine Learning and advanced filtering to maintain accurate navigation without a GPS signal.

The architecture consists of three core pillars:

1. **AI Motion Estimator (TCN):** We use a Temporal Convolutional Network (TCN) with 49,665 parameters. Instead of manually integrating noisy accelerometer data, the TCN looks at a sliding window of 6-axis IMU data (accelerometer + gyroscope) and uses causal dilations to "learn" the vibration patterns of human driving, directly predicting the vehicle's forward velocity in m/s.
 2. **Sensor Fusion (Planar EKF):** We utilize an Error-State Kalman Filter (ESKF). While GPS is available, the filter trusts it. The moment GPS drops, the EKF seamlessly switches to integrating the TCN's velocity predictions and the gyroscope's yaw rate, providing a smooth, continuous trajectory.
 3. **Map Matching (Road HMM):** To prevent long-term drift, we use a Hidden Markov Model combined with the Viterbi algorithm. This algorithm snaps the dead-reckoned trajectory to the nearest logical road segments provided by OpenStreetMap (OSM) geometry.
-

3. Detailed Project Progress (Work Completed)

The project has evolved from a conceptual visual prototype into a rigorous, scientifically validated engineering pipeline.

A. The Evaluation Workbench (Web Dashboard)

- **Ablation Testing:** Built a fully interactive browser dashboard that replays physical sensor recordings (`navigation-workbench.js`).
- **Real-Time Visuals:** Integrated Leaflet.js to plot the dead-reckoned trajectory on a live map alongside the Ground Truth path.

- **Metrics:** Implemented side-by-side performance metrics (Mean Absolute Error, drift distance) allowing the team to toggle the EKF, HMM, and TCN on and off dynamically to prove the mathematical value of each component independently.

B. The Native Android Implementation

- **Kotlin Porting:** The entire Javascript navigation engine was successfully ported to Kotlin (`native/android/navigation`).
- **Mathematical Parity:** Built rigorous unit tests (`BrowserParityTest.kt`) proving that the Kotlin native code achieves bit-for-bit identical mathematical outputs to the browser prototype.
- **App Infrastructure:** Built an Android background service (`NavigationService.kt`) capable of capturing physical IMU and GNSS data on a physical phone, complete with session cataloging and local JSON storage.

C. The Machine Learning Scaffold

- **TCN Architecture:** Designed the 1D PyTorch TCN model (`ml/model.py`) optimized for extreme mobile edge performance (fp32).
- **Data Splitting (Leakage Prevention):** Built a highly complex dataset splitter (`ml/split.py`) that uses connected-component graphs to group trips by Vehicle, Phone, and Route. This guarantees that a specific car's data never leaks across the Train/Test sets, ensuring the AI's accuracy is mathematically honest.
- **Hardware Autotuning:** Engineered a GPU memory probe (`ml/memory_probe.py`) that successfully connected to an NVIDIA RTX 4050 Laptop GPU, verifying it can train the model within the strict 6 GB VRAM hardware constraint using gradient accumulation (batch size 128, 8 threads).

D. The Physical Data Pipeline

- **Dataset Retrieval:** Built custom scripts (`tools/fetch_io_vnbd_lfs.py`) that bypassed Git LFS limits to successfully download the 1.67 GB `IO-VNBD` physical driving dataset.
- **Inventory Scanning:** Built a scanner (`tools/inventory_io_vnbd.py`) that hashed all 564 CSV files, identified 230 duplicate payloads, and analyzed the messy clock offsets and header structures of the raw data.

4. Current State & Remaining Work

The entire software infrastructure (Native App, ML Pipeline, Web Dashboard) is **100% complete and tested**. The final remaining task is the **Human Data Review**.

Because physical sensor data is messy, a human must manually open the `IO-VNBD` CSV files, define the clock offset between the phone and the vehicle, and write a small "adapter script" to apply gravity-based frame corrections (rotating the phone's axes to align with the vehicle's forward motion). Once that adapter script cleans the data, it will be fed into our completed `split.py` and `train.py` scripts to finalize the AI model.